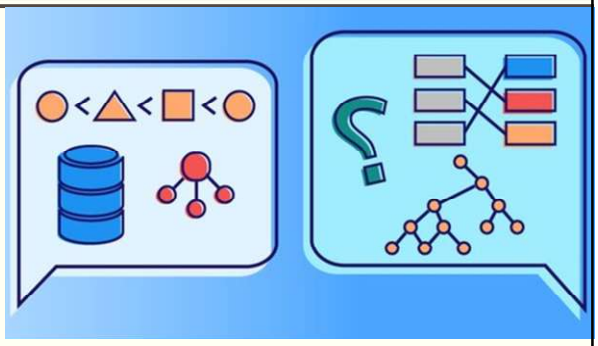
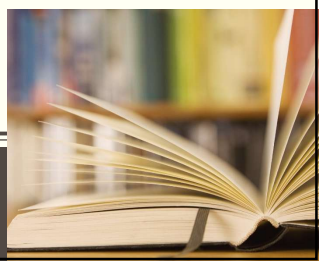
ALGORITMOS Y ESTRUCTURAS DE DATOS



Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 26/10

- Archivos binarios: Función de escritura.
- Función de Posicionamiento: Seek.
- Posición del marcador de posicionamiento: Tell.

2

Algoritmos y Estructuras de Datos – AEDD – Asistencia 26/10

Contraseña del estudiante

skc3wy

3

Archivos

- **Según donde se almacenan**
  - **Internas** (en memoria principal)
  - **Externas** (en memoria auxiliar)
- **Según tipos de datos de sus componentes**
  - **Homogéneas** (todas del mismo tipo)
  - **No homogéneas** (pueden ser de distinto tipo)
- **Según la implementación**
  - **Provistas por los lenguajes** (básicas)
  - **Abstractas** (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- **Según la forma de almacenamiento**
  - **Estáticas** (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
  - **Dinámicas** (su tamaño varía durante el módulo y sus posiciones también)

4

Opciones para abrir archivos

Para abrir un archivo usamos el método:

void open (const char nombre, int nModo = ios: :in)*

- nombre es el nombre del archivo
- nModo es el modo de apertura.

Modo	Significado
`ios::in`	Abre para lectura (valor por defecto para `ifstream`). Se sitúa al principio.
`ios::out`	Abre para crear nuevo archivo (valor por defecto para `ofstream`). Se sitúa al principio y pierde los datos.
`ios::app`	Abre para añadir al final.
`ios::trunc`	Crea un archivo para escribir/leer (si ya existe se pierden los datos).
`ios::binary`	Archivo binario.

5

Archivos binarios: la función de lectura

Prototipo: istream& read (char* s, streamsize n);

Lee un bloque de datos desde un archivo binario.

Extrae n caracteres (indicados en el 2do parámetro) desde un archivo de entrada y los almacena en la cadena de caracteres s (indicada en el 1er parámetro).

✓ Para leer un tipo de dato que no sea una cadena de caracteres se deberá hacer un casting.

✓ Esta función copia un bloque de datos sin chequear su contenido (si tiene caracters nulos o de final).

Cuando no sabemos la medida en bytes de la información que queremos leer, deberemos usar la función sizeof que dado un tipo nos devolverá su tamaño en bytes.

Ejemplo: Queremos leer de un archivo un valor de tipo double y guardarlo en una variable var de tipo double. Haremos:

archivo.read ((char*)(& var), sizeof(double));

6

Archivos binarios: la función de escritura

Prototipo: ostream& write (const char* s, streamsize n);

Escribe un bloque de datos en un archivo binario.

Inserta n caracteres (indicados en el 2do parámetro) en el archivo de salida, desde la cadena de caracteres s (indicada en el 1er parámetro).

✓Esta función tampoco valida contenido.

Ejemplo: Queremos escribir en un archivo un valor de tipo double que tenemos almacenado en una variable var de tipo double. Haremos:

archivo.write ((char)(& var), sizeof(double));*

7

Ejercicio 1

Mediante un programa C++, escribir un archivo binario llamado grupo.dat, formado por un conjunto de 200 pares de números generados aleatoriamente. Cada par de datos se conforma por un flotante y un entero.

8

Función de posicionamiento

En el acceso aleatorio (directo), cualquier carácter en el archivo abierto puede leerse en forma directa sin tener que leer primero, en forma secuencial, todos los caracteres almacenados antes que él.

- Para proporcionar acceso aleatorio a los archivos, cada objeto ifstream crea en forma automática un marcador de posición de archivo, seekg/seekp.
- Este marcador es un numero entero largo que representa un desplazamiento desde el principio de cada archivo e indica el lugar desde donde se va a leer o a escribir el siguiente carácter.

9

Función de posicionamiento: seek

Para leer y escribir archivos de acceso directo usaremos las operaciones read y write, y tenemos disponible la instrucción que nos permite posicionarnos en una posición aleatoria en el archivo:

seekg : "seek get", se puede aplicar sobre archivos de entrada, así como archivos de E/S, e indica la posición del próximo get.

Para archivos de salida, la posición para el próximo put se puede indicar con: seekp : "seek put",

Ambas tienen el mismo formato:

Posición donde se hará la proxima lectura

seekg(pos_type* posi) -> (Esta forma es siempre relativa al inicio del archivo)

O bien:

Cantidad off de posiciones a desplazarse

Desde donde se desplaza

seekg(off_type* off, ios_base: :direcc).

Lugares desde donde se van a realizarlos desplazamientos

Valor	Descripción
ios::beg	El desplazamiento es relativo al inicio del fichero.
ios::end	El desplazamiento es relativo al final del fichero.
ios::cur	El desplazamiento es relativo a la posición actual en el fichero.

10

Función de posicionamiento: Seek

La función seek nos permite acceder a cualquier posición del archivo, no tiene por qué ser exactamente al principio de un registro.

Cada carácter en un archivo de datos se localiza por su posición en el archivo. El primer carácter en el archivo se localiza en la posición 0, el siguiente en la posición 1, etc. Se hace referencia a la posición de un carácter como su desplazamiento desde el inicio del archivo. Por tanto, el primer carácter tiene un desplazamiento de 0, el segundo carácter tiene un desplazamiento de 1, etc., para cada carácter en el archivo.

El desplazamiento de la función seek es de 1 byte.

Un desplazamiento positivo significa avanzar en el archivo y un desplazamiento negativo significa retroceder.

11

Función de posicionamiento: Seek

Por ejemplo:

- Si nos queremos desplazar al inicio del archivo haremos seekg(0),

- Si nos queremos desplazar al final del archivo: seekg (0, ios:: end).

Hay que tener en cuenta que el final de un archivo no es la posición del último elemento sino la posición del cursor después de leer el último elemento.

Esto quiere decir que si queremos consultar el último elemento tendríamos que ponernos en la posición: seekg(-1, ios::end).



12

Posición del marcador de posicionamiento: Tell

Para saber, en un momento determinado, la posición en la que estamos, ésta se puede consultar con la instrucción tell

tellg () : devuelve la posición del cursor (o el valor de desplazamiento del marcador de posición) dentro del archivo de lectura.

Para los archivos de escritura tendremos **tellp ()** que funciona igual.

Ejemplo: si se leyeron diez caracteres desde un archivo de entrada llamado archivo_entr, la siguiente sentencia devolverá el número entero 10:

```
archivo_entr.tellg();
```

Esto significa que el siguiente carácter que se va a leer está desplazado diez posiciones desde el inicio del archivo, y es el undécimo carácter en el archivo.

13

Archivo de texto y archivo binario

Se tiene un arreglo de enteros con valores grandes: int numeros[] = {123456, 789012, 345678, 901234}. Se pide guardarlos en un archivo de texto y en uno binario para comparar cómo se delimitan y el peso final.

14

Ejercicio 2

Escribir un programa que abra el archivo generado en el ejercicio anterior, y solicite al usuario que ingrese un flotante, un entero y una posición.

El programa debe sobreescribir el par en la posición ingresada por el usuario, por el nuevo par.

Luego, debe mostrar la lista de datos por pantalla mostrando un par por línea.

15

Ejemplo de escritura en archivo binario

```
#include <iostream>
#include <fstream>
using namespace std;

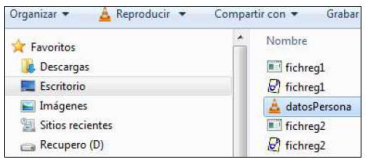
typedef char id[20];

struct persona {
    id nombre, apellido;
    int edad;
};

int main() {
    ofstream fPers;
    fPers.open("datosPersona.bin");
    if(!fPers)
        cout << "Error al abrir el fichero" << endl;
    else {
        persona p;
        strcpy(p.nombre, "Agustina");
        strcpy(p.apellido, "Soler");
        p.edad = 34;

        // Escribimos una persona
        fPers.write((char *)(& p), sizeof(p) );

        fPers.close();
    }
    return 0;
}
```



Graba en el archivo de salida una cantidad de bytes (indicada por sizeof), obtenida de la dirección de memoria (&p) donde están guardados los datos que se escribirán en el archivo, tratándolos como una cadena de caracteres (puntero a char)

16

Ejemplo de lectura/escritura de arreglos

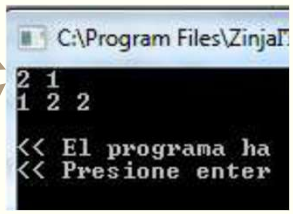
```
int main(void) {
    int a[3][4]={{1,2,3,4},{1,1,1,1},{2,2,2,2}};
    int b[3][4];
    int c[12];

    //--------------------------------
    ofstream f;
    f.open ("bufl.bin");

    f.write ( (char*)a, sizeof(a) );
    f.close( );
    //---------------------------------
    ifstream g;
    g.open ("bufl.bin");

    g.read ( (char*) b, sizeof(b) );
    cout << b[2][3]<< " " << b[0][0] << endl;
    g.close( );
    // ---------------------------------
    ifstream h;
    h.open ("bufl.bin");

    h.read ( (char*) c, sizeof(c) );
    cout << c[5] << " " << c[10] << " " << c[11];
    h.close( );
    return 0;
}
```



17

Ejemplo de fusión de dos ficheros

```
#include <iostream>
#include <fstream>
    using namespace std;
void fusionarFicheros(ifstream& f1, ifstream& f2, ofstream& fResu);

int main()
{   char nombre1[100] , nombre2[100];
    cout << "Introduce el nombre del primer fichero: ";
    cin >> nombre1;
    ifstream f1;
    f1.open(nombre1);
    cout <<"Introduce el nombre del segundo fichero: ";
    cin >> nombre2;
    ifstream f2;
    f2.open(nombre2);;
    char nombreDest[100];
    cout << "Introduce el nombre del fichero destino: ";
    cin >> nombreDest;
    ofstream fDest;
    fDest.open(nombreDest);
    fusionarFicheros(f1, f2, fDest);
    f1.close ();
    f2.close ();
    fDest. close ( );
    return 0;
}
```

18

Ejemplo de fusión de dos ficheros

```
void fusionarFicheros(ifstream& f1, ifstream& f2,ofstream& fDest)
{
    char e;
    f1.unsetf(ios::skipws);
    // Ponemos enfDest el contenido del primer fichero
    while (f1 >> e)
        fDest << e;
    f2.unsetf(ios::skipws);
    // Ponemos enfDest el contenido del segundo fichero
    while (f2 >> e)
        fDest << e;
}
```

19

Ejemplo de fusión de dos ficheros

```
void fusionarFicheros(ifstream& f1, ifstream& f2,ofstream& fDest)
{
    char e;
    f1.unsetf(ios::skipws);
    // Ponemos enfDest el contenido del primer fichero
    while (f1 >> e)
        fDest << e;
    f2.unsetf(ios::skipws);
    // Ponemos enfDest el contenido del segundo fichero
    while (f2 >> e)
        fDest << e;
}
```

Cualquier bandera establecida puede desactivarse.
Para desactivar una bandera usamos la función unsetf.
Cuando la bandera de formato skipws se establece, los espacios en blanco seguidos se leen y se descartan del flujo hasta leer algo que no sea blanco.
Esto se aplica a todas las operaciones de entrada con formato realizado con el operador >> en la secuencia.
Espacios de tabulación, de retornos de carro y espacios en blanco son todos considerados espacios en blanco (ver isspace).
Para flujos estándares, la bandera skipws se encuentra en la inicialización.

20

Ejercicio de Transacciones



Funcionalidad:

- actualiza las cuentas existentes,
- agrega nuevas cuentas,
- elimina cuentas, y
- guarda un listado con formato de todas las cuentas actuales en un archivo de texto

21

Ejercicio de Transacciones

```
#include <stdio.h>
#include <fstream>
#include <iostream>
#include <iomanip>
using namespace std;


// clientData structure definition
struct clientData {
    unsigned int acctNum; // account number
    char lastName[ 15 ]; // account last name
    char firstName[ 10 ]; // account first name
    double balance; // account balance
};
typedef struct clientData cliente;

// prototypes
unsigned int enterChoice();
void textFile (fstream &);
void updateRecord( fstream & );
void newRecord( fstream & );
void deleteRecord( fstream & );
void mostrarLinea (ostream&, const cliente &);
int obtenerCuenta (const char * );
```

22

Ejercicio de Transacciones

```
int main( void )
{
    fstream credito;
    credito.open("credit.dat", ios::in|ios::out);
        if ( !credito) {
        cout << "File could not be opened." ;
    }
    else {
        unsigned int choice; // user's choice// enable user to specify action
        while ( ( choice = enterChoice() ) != 5 ) {
            switch ( choice ) {
            case 1:
                textFile( credito );break;
            case 2:
                updateRecord( credito );break;
            case 3:
                newRecord( credito );break;
            case 4:
                deleteRecord( credito );break;
            default:
                cout << "Incorrect choice" ;break;
            }
        }
    credito.close();
    }
} // end main
```

23

Ejercicio de Transacciones

```
unsigned int enterChoice( void )
{
    unsigned int menuChoice;
    cout <<"\nEnter your choice\n"
        <<"1 - store a formatted text file of accounts called\n"
        <<"    \"accounts.txt\" for printing\n"
        <<"2 - update an account\n"
        <<"3 - add a new account\n"
        <<"4 - delete an account\n"
        <<"5 - end program\n? " ;

    cin >> menuChoice;
    return menuChoice;
}
```

24

Ejercicio de Transacciones

```
// create formatted text file for printing
void textFile( fstream & leerDeArchivo)
{
    ofstream salida; // accounts.txt file pointer
    cliente client ;
    salida.open( "accounts.txt", ios::out);
    if (!salida){
        cout << "File could not be opened." ;
    }
    else {
        salida << left << setw(10) << "Acct"<<setw(16) <<"Last Name"
            << setw(14)<<"First Name"<< right <<setw(10) <<"Balance"<<endl;

        leerDeArchivo.seekg (0);
        leerDeArchivo.read((char *) &client, sizeof(cliente));

        while ( !leerDeArchivo.eof() ) {
            mostrarLinea (salida,client);
            leerDeArchivo.read((char *) &client, sizeof(cliente));
            }

        leerDeArchivo.close();
    }
} // end function textFile
```

Se escriben los títulos del listado

25

Ejercicio de Transacciones

```
void mostrarLinea( ostream & salida, const cliente& registro)
{
    salida << left << setw(10) << registro.acctNum<<setw(16) <<registro.lastName
        << setw(11)<< registro.firstName << right <<setw(10) <<registro.balance<<endl;
}

int obtenerCuenta (const char * indicador)
{
    int num;
    do {
        cout << indicador <<"(1-100): ";
        cin >> num;
    } while (num <1 || num > 100);
    return num;
}
```

26

Ejercicio de Transacciones

```
// update balance in record
void updateRecord( fstream & actualizarArch )
{
    unsigned int account = obtenerCuenta ("Escriba la cuenta que desea actualizar");
    double transaction;
    // create clientData with no information
    cliente client = { 0, "", "", 0.0 };

    actualizarArch.seekg ( ( account - 1 ) * sizeof(cliente));
    actualizarArch.read((char *) &client, sizeof(cliente));

    if ( client.acctNum  != 0 ) {
        mostrarLinea (cout,client);

        cout <<endl <<"Enter charge ( + ) or payment ( - ): " ;
        cin >>transaction ;

        client.balance += transaction; // update record balance

        actualizarArch.seekp ( ( account - 1 ) * sizeof(cliente));
        actualizarArch.write((char *) &client, sizeof(cliente));
        mostrarLinea (cout,client);
        }
    else cout << "La cuenta " << account <<" no tiene informacion" << endl;

}   // create and insert record
```

27

Ejercicio de Transacciones

```
void newRecord( fstream & insertarEnArch)
{
    unsigned int account = obtenerCuenta ("Escriba la cuenta que desea actualizar");
    insertarEnArch.seekg ( ( account - 1 ) * sizeof(cliente));
    cliente client ;
    insertarEnArch.read((char *) &client, sizeof(cliente));
    if ( client.acctNum != 0 ) {

    cout<< " Escriba apellido, nombre y saldo" << endl;
    cin >> client.lastName >> client.firstName>> client.balance ;

    insertarEnArch.seekp ( ( account - 1 ) * sizeof(cliente));
    insertarEnArch.write((char *) &client, sizeof(cliente));
        }
    else cout << "La cuenta " << account <<" ya tiene informacion" << endl;
}                                   client.acctNum != 0
```

28

Ejercicio de Transacciones

```
void deleteRecord( fstream & eliminarEnArch)
{
    unsigned int account = obtenerCuenta ("Escriba la cuenta que desea eliminar");
    eliminarEnArch.seekg ( ( account - 1 ) * sizeof(cliente));
    cliente client ;
    eliminarEnArch.read((char *) &client, sizeof(cliente));
    if ( client.acctNum != 0 ) {
        cliente client = { 0, "", "", 0.0 };
        eliminarEnArch.seekp ( ( account - 1 ) * sizeof(cliente));
        eliminarEnArch.write((char *) &client, sizeof(cliente));
        cout << " La cuenta"<<account << "ha sido eliminada";
    }
    else cout << "La cuenta " << account <<" esta vacia" << endl;
}
```

29

Algoritmos y Estructuras de Datos – AEDD – Asistencia 26/10

Contraseña del estudiante

skc3wy

30

Para profundizar

Capítulo 17

"Procesamiento de archivos"

Libro: "Cómo programar en C++" – Deitel & Deitel,

pág. 708

Capítulo 8

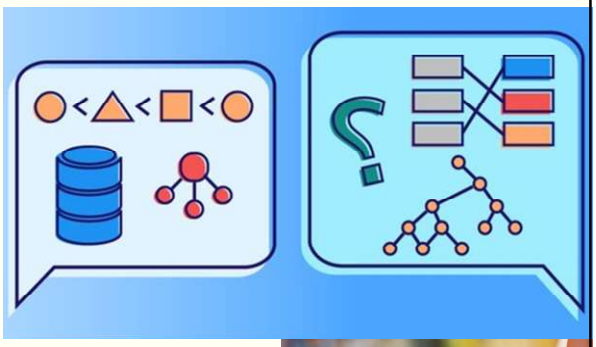
"Flujos de archivos de E/S y archivos de datos"

Libro: "C++ para ingeniería y ciencias" 2da edición -

Gary J. Bronson, pág. 443

31

ALGORITMOS Y ESTRUCTURA DE DATOS



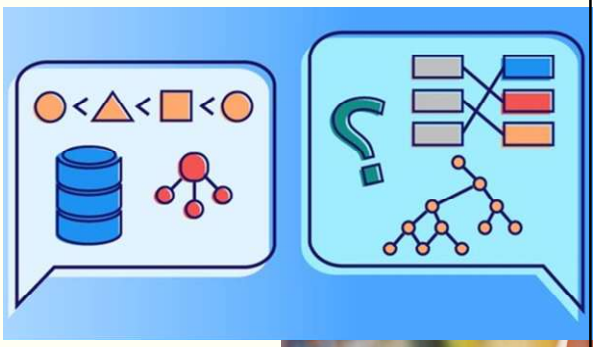
Comisión F – 2026 – Cursado Anual

¿DUDAS?



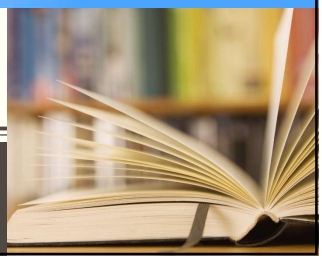
32

ALGORITMOS Y ESTRUCTURA DE DATOS



Comisión F – 2026 – Cursado Anual

¡Muy buena semana!



33